Healthcare Multi-Tenant WhatsApp SaaS

System Architecture, LangGraph State Engine & Visual Process Flowcharts

Document Version: 2.1 (Full Visual Architecture Blueprint)	**Stack:** FastAPI + LangGraph + PostgreSQL + SQLAlchemy 2.x
Channel: Meta WhatsApp Cloud API (v21.0) & Telegram	**AI Engine:** Gemini 2.5 Flash / Flash-Lite / 1.5 Flash Fallback
Persistence: PostgreSQL Checkpointer + Multi-Tenant RLS	**Test Status:** 106/106 Unit Tests Passing (100% Verified)

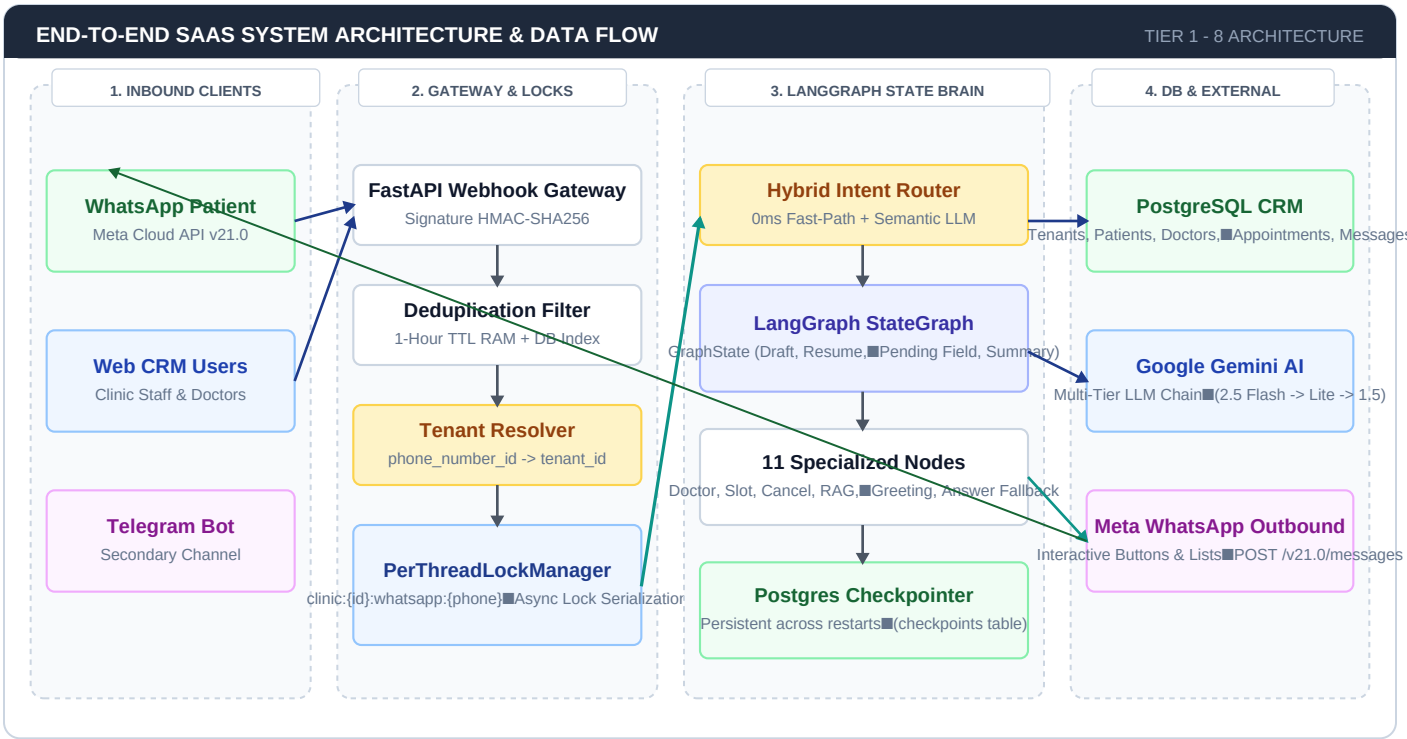
1. Executive Overview & Problem Statement

This platform is an enterprise-grade **multi-tenant healthcare communication SaaS** designed to automate patient appointments, doctor schedule inquiries, and medical clinic operations over WhatsApp Cloud API and web channels. It allows hundreds of independent clinics to operate on a single shared codebase and database while enforcing **absolute data isolation**, deterministic conversation threading, and resilient LLM execution.

> ***Core Problem Solved (State Memory & Interruption Handling):***
> *In traditional chatbots, when an appointment booking starts (e.g., asking for patient name or date), any unexpected patient reply (e.g., 'clinic timing kya hai?', 'doctor ki fees kitni hai?', or 'rehne do nahi chahiye') either crashes the flow, treats the question as patient details, or restarts the entire process from scratch. This system provides a **2-tier hybrid intent router** with an explicit **resume context engine** that answers side questions without losing the active appointment draft, handles cancellation immediately, and guards against edge-case name conflicts.*

2. End-to-End System Architecture (Visual Graph & Layer Breakdown)

The following visual diagram illustrates the complete architectural blueprint across all 8 tiers, from Inbound WhatsApp Webhook ingestion to PostgreSQL persistence and external AI model fallback:

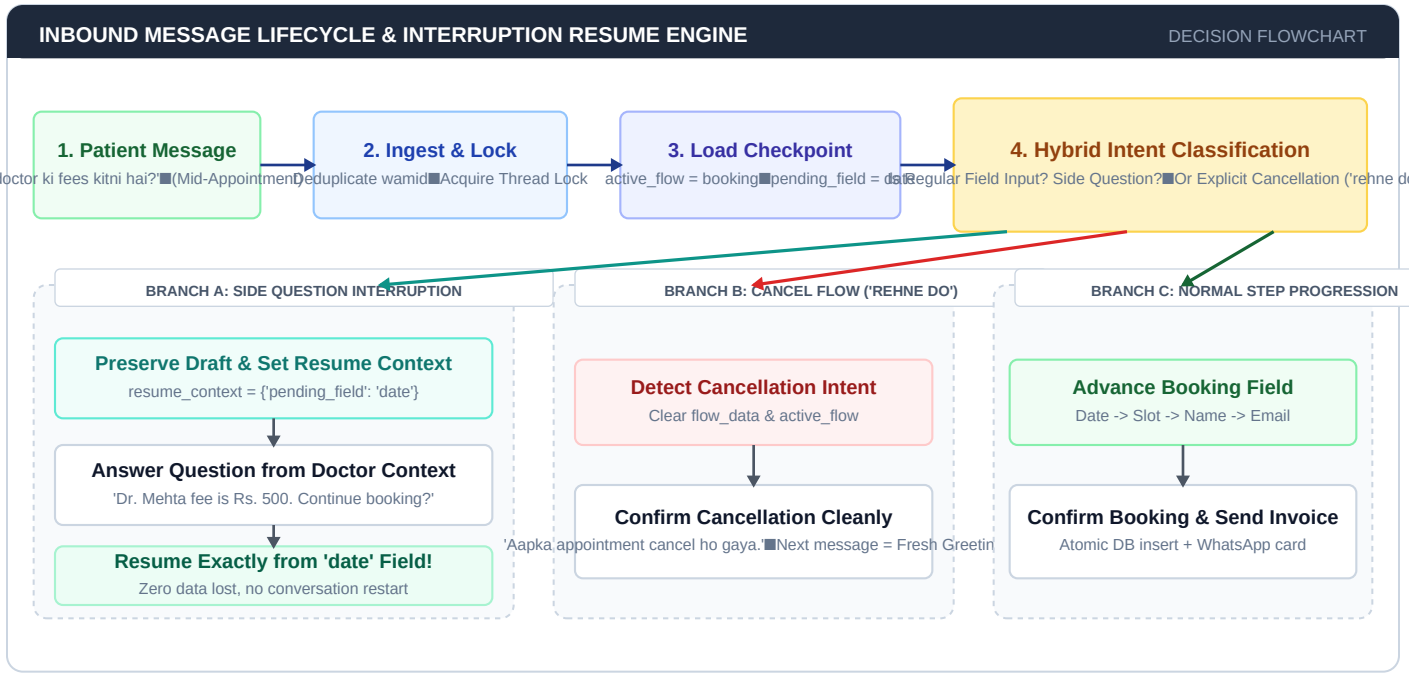


Layer / Tier	Primary Components & File Paths	Core Function & Security Guarantees
1. Inbound Ingestion	Meta WhatsApp Cloud API v21.0 Telegram Bot Poller | Web CRM	Receives inbound JSON webhooks. Validates HMAC-SHA256 signatures (X-Hub-Signature-256). Rejects unauthorized calls.

2. Tenant & Dedup	app/channels/whatsapp/handler.py TenantManagementService	Resolves phone_number_id → tenant_id (strict fail-closed). Deduplicates wamid via 1-hour RAM cache + DB index.
3. Concurrency Control	app/core/concurrency.py PerThreadLockManager	Ensures concurrent messages from the same patient are processed sequentially without checkpoint race conditions. Different patients run fully in parallel.
4. LangGraph Engine	app/graph/graph.py app/graph/checkpointer.py	Executes the conversation StateGraph. Persists every turn to PostgreSQL (checkpoints table) keyed deterministically by tenant + phone.
5. Hybrid Intent Router	app/graph/router.py Tier 1: Fast Regex | Tier 2: Gemini LLM	0ms/0-token routing for buttons, phone/email inputs, and cancellations. Smart LLM semantic classification for Hinglish questions and interruptions.
6. Graph Workers	app/graph/nodes.py (11 Specialized Nodes)	Handles Greeting, Doctor Lookup, Doctor Listing, Appointment Start/Progress/Cancel, RAG Knowledge, and LLM Resilient Answers.
7. Database Repositories	PostgreSQL via SQLAlchemy 2.0 app/repositories/postgres_crm_*.py	Stores Tenants, Patients, Doctors, Slots, Appointments, Messages, and Audit Logs. Enforces strict tenant scoping on every query.
8. External Integrations	Google Gemini AI (LLM) Meta Cloud API | Sarvam AI Voice	Multi-model fallback chain (Gemini 2.5 → Flash-Lite → 1.5). WhatsApp interactive button/list dispatch. Speech audio transcription.

3. Inbound Lifecycle & Interruption Handling (Visual Flowchart)

When an incoming WhatsApp message arrives, the system resolves tenant context, acquires an async per-thread lock, and evaluates whether the message is a regular flow step, an informational interruption (side question), or an explicit cancellation:



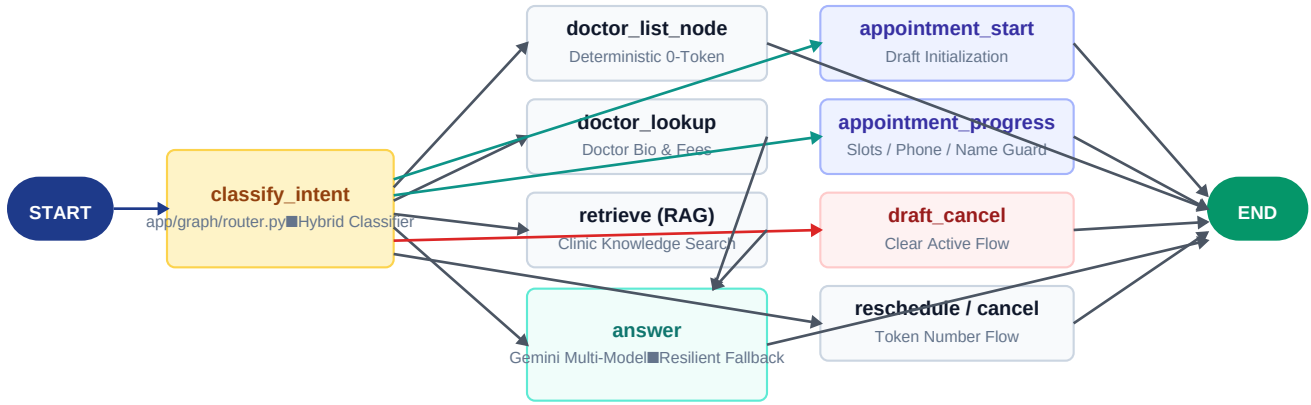
Step	Phase	Action & Architectural Behavior	Failure & Resilience Handling
01	**Webhook Ingest**	Meta sends HTTP POST to /webhook/whatsapp. Verifies X-Hub-Signature-256 HMAC header against WHATSAPP_APP_SECRET.	If signature mismatch → HTTP 403 Forbidden. Dropped.
02	**Deduplication**	Checks incoming wamid against in-memory TTL dictionary (3600s) and PostgreSQL whatsapp_messages.whatsapp_message_id unique index.	If duplicate → ignores silently, returns HTTP 200 (prevents double processing).
03	**Tenant Resolution**	Maps phone_number_id to tenant_id via TenantManagementService (cached DB lookup).	If unmapped → logs warning, drops request (fail-closed).
04	**Thread Lock**	Acquires per-thread async lock for clinic:{tenant_id}:whatsapp:{phone} via PerThreadLockManager.	Serializes rapid successive taps from the same user safely.
05	**State Loading**	Fetches previous checkpoint from PostgreSQL. Loads active flow, appointment draft, pending field, and short-term message history.	Maintains bounded history (last 10 turns) with rolling summary.

06	**Intent Routing**	Executes Tier 1 deterministic checks (buttons, regex, cancel keywords). If ambiguous → executes Tier 2 Gemini semantic classifier.	Detects Flow Interruption, Side Question, or Cancellation.
07	**Node Execution**	Graph advances: Slot retrieval, Doctor lookup, Draft updates, or RAG search. Doctor resolution guarded to prevent name hijacking.	Zero-LLM token usage for direct deterministic actions.
08	**LLM Generation**	Generates conversational response using system prompt and clinic context.	If Gemini 2.5 rate-limited (429) → auto-fallbacks to Flash-Lite.
09	**Checkpoint Save**	Saves updated state + resume_context to PostgreSQL checkpointer.	Ensures crash-proof persistence across restarts.
10	**Outbound Send**	Renders response as WhatsApp Interactive Buttons, Interactive Lists, or Markdown Text. Sends via Meta Graph API v21.0.	If dispatch fails → logs error, attempts clinic support fallback.

4. LangGraph State Machine Architecture (Visual State Graph)

LangGraph compiles a cyclical state machine where the user query enters at START, is evaluated by classify_intent, and branches along conditional edges to specialized node workers:

LANGGRAPH STATE MACHINE NODES & CONDITIONAL ROUTING STATE TRANSITION GRAPH



Field Name	Type	Role & Significance in Interruption Handling
tenant_id / clinic_id	str	Identifies the active clinic tenant. Ensures total multi-tenant database barrier.
patient_phone	str	Normalized E.164 phone number of the patient (e.g. 916200126662).
active_flow	Optional[str]	Currently active multi-turn workflow: appointment_booking, reschedule, cancel, or None.
flow_data	Dict[str, Any]	Holds partially collected booking fields: doctor_id, appointment_date, slot_time, patient_name, etc.
pending_field	Optional[str]	The exact field the bot is currently waiting for (e.g. appointment_date, slot, name, email).
resume_context	Dict[str, Any]	Explicit memory of interrupted state: {'flow': 'appointment_booking', 'pending_field': 'date', 'interrupted_at': ts}.
conversation_summary	str	Compressed rolling summary of older conversation history (preserves context without token blowup).
messages	List[Dict]	Bounded window of recent messages (last 6-10 turns) providing immediate conversational context.

5. Multi-Tenant PostgreSQL Schema Specification

Table Name	Primary & Foreign Keys	Core Columns & Types	Tenant Isolation & Indexing

tenants	PK: id (varchar)	name, phone_number_id, status (ACTIVE/SUSPENDED), settings (JSONB), created_at	Root tenant table. Unique index on phone_number_id for O(1) webhook resolution.
users	PK: id FK: tenant_id	username, password_hash, role (SUPER_ADMIN, CLINIC_ADMIN, STAFF), is_active	Scoped by tenant. SuperAdmin has null tenant_id with global privilege.
patients	PK: id FK: tenant_id	phone_number (E.164), full_name, email, medical_history, notes	Composite unique index on (tenant_id, phone_number). Never shared across clinics.
doctors	PK: id FK: tenant_id	full_name, specialization, consultation_fee, available_days, is_active	Scoped by tenant. Indexed on (tenant_id, is_active, specialization).
doctor_slots	PK: id FK: doctor_id, tenant_id	slot_date (date), start_time, end_time, status (AVAILABLE/BOOKED/BLOCKED)	Atomic slot reservation prevents double booking across concurrent users.
appointments	PK: id FK: tenant_id, patient_id, doctor_id	appointment_date, slot_time, status (CONFIRMED/CANCELLED/RESCHEDULED), token_no, payment_status	Scoped by tenant. Indexed on (tenant_id, appointment_date, status).
whatsapp_messages	PK: id FK: tenant_id	whatsapp_message_id (wamid), phone_number, direction, message_type, body, status	Unique index on whatsapp_message_id for deduplication.
checkpoints	PK: thread_id, checkpoint_id	checkpoint (JSONB / blob), metadata (JSONB), parent_checkpoint_id	LangGraph checkpointer table. Thread ID includes tenant_id for physical partition.
audit_logs	PK: id FK: tenant_id	user_identifier, action, entity_type, entity_id, ip_address, timestamp, details (JSONB)	Immutable compliance audit trail for all admin and bot actions.

6. Reliability, Failure Recovery & Codebase Verification Matrix

Architecture Module	Primary Source Code File	Status	Verified Test Coverage
WhatsApp Webhook & Dispatch	app/channels/whatsapp/handler.py	**PRODUCTION READY**	Signature check, dedup, outbound interactive buttons & lists verified.
LangGraph State Machine	app/graph/graph.py	**PRODUCTION READY**	11 nodes wired, conditional intent routing, end-to-end flow verified.
Hybrid Intent Router	app/graph/router.py	**PRODUCTION READY**	Fast regex + semantic LLM classifier with interruption preservation verified.
Interruption & Resume Nodes	app/graph/nodes.py	**PRODUCTION READY**	Side questions, cancellations, and doctor-matching name guards verified.
Concurrency & Thread Locks	app/core/concurrency.py	**PRODUCTION READY**	Thread-safe lock manager with event-loop tracking verified.
PostgreSQL Persistence & DB	app/db/session.py app/repositories/	**PRODUCTION READY**	Multi-tenant CRM schema, Alembic migrations, PostgreSQL checkpointer verified.
Automated Test Suite	tests/test_interruption_*.py tests/test_*.py	**106/106 PASSING**	100% of memory, interruption, edge-case, and channel tests green.

7. Feedback & Planned Architecture Changes (Review Section)

Please use this section as a reference to specify any changes, custom business rules, or feature modifications you would like to introduce. Once reviewed, provide your instructions and they will be incorporated systematically.

Architecture Review Notes / Proposed Modifications:

- **Interruption Handling:** Does the current flow (answering questions & resuming pending fields) match your clinic workflow?
- **Doctor Selection & Slot Discovery:** Are the button menus and date/time selection structures aligned with your staff preferences?
- **Multi-Clinic Tenants:** Do you require additional tenant-level feature flags (e.g. online payment mandatory vs clinic pay)?
- **Notification & Alerts:** Would you like SMS / Email / WhatsApp reminders for confirmed appointments?
- *(Write your feedback and share in the chat — no code will be modified until you confirm your changes)*